

Security Audit

ForeGate (Prediction Market)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	32
Conclusion	33
Our Methodology	34
Disclaimers	36
About Hashlock	37

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The ForeGate team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

ForeGate is a next-generation prediction market platform, empowering users to forecast real-world events and trade on their outcomes. Whether you're a professional trader, industry expert, or just someone who follows trends, ForeGate offers a transparent, data-driven, and engaging marketplace where your insights can translate into real earnings.

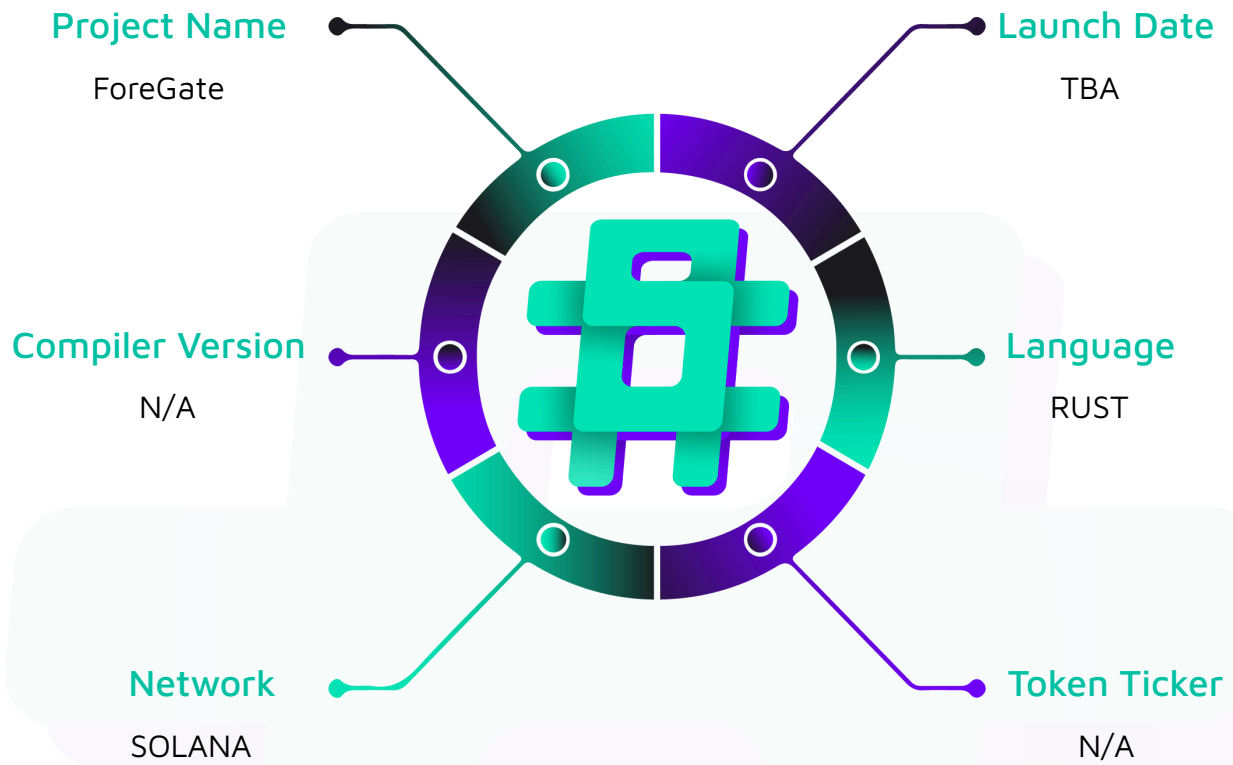
Project Name: ForeGate

Project Type: Prediction Market

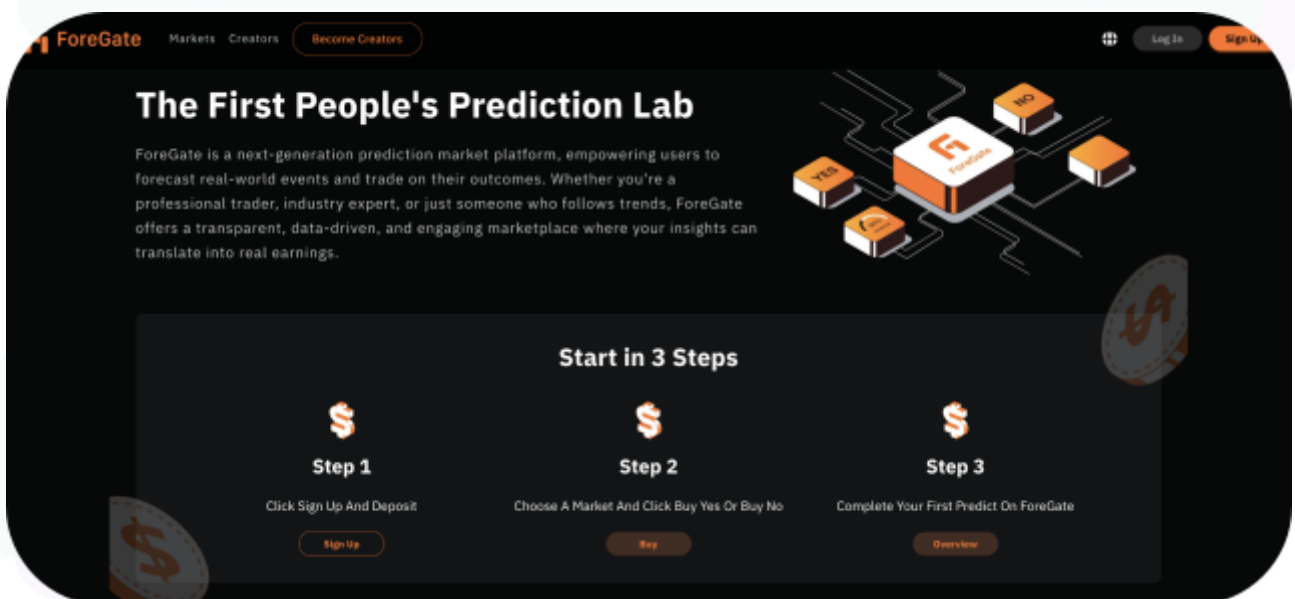
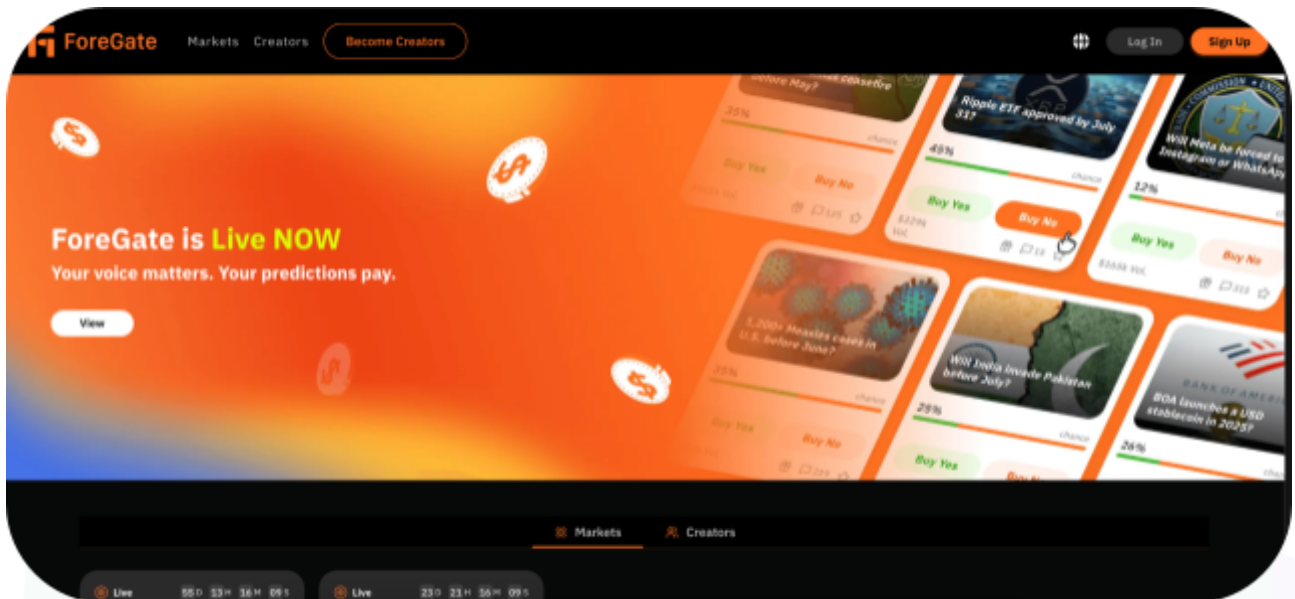
Website:

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the ForeGate project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	ForeGate Smart Contracts
Platform	Solana / Rust
Audit Date	September, 2025
GitHub repo	https://github.com/gph-tech/foregate-cfmm-anchor
Contract	programs/cfmm-prediction-market
Audited GitHub Commit Hash	7f1a23e8d49c2b37a2788386f087cb32d6694c05
Fix Review GitHub Commit Hash	b41a19b8

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

4 Medium severity vulnerabilities

6 Low severity vulnerabilities

1 Gas Optimisations

5 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
programs/cfmm-prediction-market - Allows the operator to: - Set operator address - Perform an emergency withdrawal - Allows the system authority to: - Claim unresolved pools - Allows users to: - Create prediction pools - Buy shares - Sell shares - Resolve pools - Claim winnings - Claim unresolved pools	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the ForeGate project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the ForeGate project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01]

[cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#create_prediction_pool](#) - Lack of canonical pool ATA enforcement allows fragmentation of pool funds

Description

When creating a prediction pool in line 1543, a canonical `pool_token_account` ATA is initialized to store collateral deposits. However, in later instructions (e.g., `BuyShares` in line 1576), the program accepts any PDA-owned ATA as the pool account without checking that it matches the original one created in the `CreatePredictionPool` instruction.

This is problematic because malicious actors can intentionally fragment pool funds across multiple accounts, causing the original `pool_token_account` not to hold the actual value recorded in `prediction_pool.total_collateral_token_deposited`.

Specifically, attackers first create many token accounts for the collateral mint, then call the SPL Token's [SetAuthority instruction](#) to transfer ownership to the `prediction_pool` PDA. This allows them to use the "fake" accounts as parameters to the `BuyShares` instruction, causing their funds to be deposited into the "fake" accounts instead of the intended `pool_token_account`.

Impact

When the admin, operator, or other users pass in the actual `pool_token_account` in the program, the balance in the account will be less than intended. This causes users to be underpaid for their winnings from `claim_winnings`, resulting in a loss of funds due to the `min(calculated_amount, pool_token_account.amount)` condition in line 1274. This issue also affects users and admins when they attempt to claim unresolved funds.

Recommendation

Store the canonical `pool_token_account` address created from `CreatePredictionPool` in the `PredictionPool` struct, and enforce a strict constraint in all relevant instructions to ensure only the canonical vault can be passed.

For example: `constraint = pool_token_account.key() == prediction_pool.pool_token_account.`

Status

Resolved

[H-02]

cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs: #claim_unresolved - ClaimUnresolved does not mark `ctx.accounts.user_position` as mutable, causing state updates to fail

Description

When calling the `ClaimUnresolved` instruction, the `ctx.accounts.user_position` account is passed without the `mut` modifier. This is problematic because the `claim_unresolved` function attempts to commit state changes via `user_position.claimed = true` and `user_position.shares_per_outcome[outcome_index as usize] = 0` after the user claimed the unresolved funds.

Since the account was not declared as `mut`, Anchor will treat it as a read-only operation and ignore state changes. This means the state updates (i.e., marking the position as claimed and zeroing shares) will not persist on-chain.

Impact

Malicious users can repeatedly call the `ClaimUnresolved` instruction to steal funds from the pool.

Recommendation

Mark the `user_position` account as mutable to persist state changes.

Status

Resolved

Medium

[M-01]

[cfmm-prediction-market/programs/cfmm-prediction-market/src/config.rs](#) -
Hardcoded privileged addresses

Description

Privileged accounts (fee receiver, base pool receiver, admin, deployer) are hardcoded as static strings. This means that changing any privileged address requires a code change and redeployment, reducing flexibility.

Impact

Any necessary key rotation or compromise requires redeployment, which is both time-consuming and prone to errors.

Recommendation

Store privileged addresses in an upgradable configuration account rather than hardcoding them in the codebase.

Status

Acknowledged

[M-02]**cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs** -

Silent overflows due to saturating_add and saturating_sub usages

Description

Throughout the codebase, several arithmetic operations are implemented using the `saturating_add()` and `saturating_sub()` functions. This is problematic because they clamp results to the numeric min/max instead of panicking or erroring:

For example:

- `a.saturating_add(b)` → returns `u64::MAX` if `a + b` would overflow.
- `a.saturating_sub(b)` → returns zero if `a < b`.

Impact

Arithmetic operations that should error will be silently ignored, which is prone to errors. For example, an overflow that occurs when increasing user shares will be silently capped, potentially resulting in incorrect share payouts.

Recommendation

Replace all `saturating_add()` and `saturating_sub()` usages with checked arithmetic.

Status

Resolved

[M-03]

[cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#resolve_pool](#) - Oracle account validation is ignored

Description

The `resolve_pool` function accepts an `oracle` account parameter, but the validation checks that enforce the oracle authenticity are commented out.

Impact

The `oracle` account can be any arbitrary account specified by the creator, which may be different than `prediction_pool.oracle`. Although the oracle key is emitted in `EVENT:POOL_RESOLVED`, its trustworthiness is not guaranteed.

Recommendation

Uncomment the validation to ensure the passed `oracle` account is `ctx.accounts.prediction_pool.oracle`.

Status

Acknowledged

[M-04]

cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#system_claim_unresolved - Missing timelock allows admin to withdraw user funds prematurely

Description

The `system_claim_unresolved` function (`SystemClaimUnresolved` instruction) allows the admin to immediately withdraw collateral from unresolved pools, provided that the `prediction_pool.deadline + TIMELOCK_INTERVAL` has elapsed and `prediction_pool.status` is 0.

The issue is that users who have purchased shares can also redeem their funds via the `ClaimUnresolved` instruction, which enforces the same validation as `SystemClaimUnresolved` did. This creates a scenario where users are unable to withdraw their positions because the admin has withdrawn their funds.

Impact

Users will lose the opportunity to withdraw their positions after the admin sweeps the pool. Additionally, the withdrawn funds are transferred to the `base_pool_receiver_token_account` and not redistributed to shareholders.

Recommendation

Introduce a grace period (e.g., 72 hours) where users can call `ClaimUnresolved` before the admin calls `SystemClaimUnresolved` to withdraw the funds.

Status

Acknowledged

Low

[L-01]

[cfmm-prediction-market/programs/cfmm-prediction-market/src/config.rs](#) -
Silent fallback to default pubkey is problematic

Description

The `fee_receiver()`, `base_pool_receiver()`, `pubkey()` and `deployer()` functions use `Pubkey::try_from(...).unwrap_or_default()`. If the provided hardcoded string cannot be parsed into a valid `Pubkey`, the function silently falls back to the default `Pubkey` value, which is `11111111111111111111111111111111`.

Impact

Errors when parsing the public keys will be silenced and ignored, resulting in an incorrect public key being parsed. For example, this may cause privilege checks that rely on the admin to be ineffective.

Recommendation

Replace `unwrap_or_default()` with an explicit failure mechanism, e.g., `.expect("invalid admin address")`.

Status

Resolved

[L-02]

cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#initialize - Possible replay due to `init_if_needed`

Description

The `initialize` instruction uses `init_if_needed` for the operator account instead of `init`. Since `init_if_needed` allows re-execution even if the account exists, this allows replay of the `initialize` instruction and repeated emission of the “CFMM Prediction Market initialized” event.

Impact

Replay of initialization events could lead to misleading or duplicate logs. Additionally, off-chain consumers may mistakenly believe multiple deployments occurred.

Recommendation

Replace `init_if_needed` with `init` to enforce one-time initialization of the operator account.

Status

Resolved

[L-03]

[cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#create_prediction_pool](#) - Possible zero shares after proportional scaling

Description

When normalizing `explicit_shares` to `initial_liquidity`, integer division may cause small weights to be truncated to zero. This may occur during skewed computations, particularly when paired with small liquidity.

For example, providing `outcome_shares` as `Some([100000, 2, 1])` and `initial_liquidity` as `1000` causes `final_outcome_shares` becomes `[1000, 0, 0]`.

Impact

Markets will become untradeable, as zero shares are not allowed in any outcomes.

Recommendation

Implement a validation that ensures `final_outcome_shares[i] > 0`, similar to lines 215-217.

Status

Resolved

[L-04]

cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#claim_winnings, claim_unresolved - State will not be committed due to return statement

Description

In the `claim_winnings` and `claim_unresolved` functions, when a user has no shares in the winning outcome, the function sets `user_position.claimed = true` before immediately returning an error.

Since returning an error reverts the transaction, the assignment to `user_position.claimed` will never be committed on-chain, rendering it ineffective.

Impact

Potential confusion and incorrect assumptions about the codebase.

Recommendation

Remove the `user_position.claimed = true` line.

Status

Resolved

[L-05]

[cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#resolve_pool](#) - Unnecessary default Pubkey check in user_position.user

Description

In `resolve_pool`, a conditional check is implemented for the `if user_position.user == Pubkey::default() { ... }`. This branch is unreachable because `user_position` must already exist as a PDA initialized via `#[account(seeds = ..., bump = user_position.bump)]`.

As such, `user_position.user` will never be the default key at this point, making this logic redundant and misleading.

Impact

Dead code increases cognitive overhead and may confuse future maintainers.

Recommendation

Remove the `if user_position.user == Pubkey::default()` branch.

Status

Resolved

[L-06]**cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs** -Missing validations for `authority_token_account`**Description**

In the `CreatePredictionPool` context struct, the `authority_token_account` account is provided, but no constraints ensure it is the associated token account (ATA) for the `authority` and the given `collateral_token_mint`.

Impact

The transaction will fail due to incorrect accounts provided.

Recommendation

Implement explicit ATA constraints to ensure `authority_token_account` truly belongs to the `authority` and uses the correct `collateral_token_mint`:

```
#[account(  
    mut,  
    associated_token::mint = collateral_token_mint,  
    associated_token::authority = authority  
)]  
pub authority_token_account: Account<'info, TokenAccount>,
```

Status

Resolved

Gas

[G-01]

`cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#initialize` - Bump can be stored in operator account

Description

The `Operator` PDA is derived with seeds = `[b"operator"]` and a bump, but only the address is stored in the `Operator` account.

Recommendation

Store the PDA bump inside the `Operator` account, and use the bump within the instructions to reduce computational power.

Status

Resolved

QA

[Q-01]

[cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs](#) -

Misleading comment regarding transfer destination

Description

In lines 1287 and 1454, the code comment states *"// Transfer tokens from pool to operator"*. However, in reality, the transfer goes to `base_pool_receiver_token_account`, not directly to the operator.

Recommendation

Update the inline comment to accurately reflect the transfer destination (e.g., *"// Transfer tokens from pool to `base_pool_receiver` account"*).

Status

Resolved

[Q-02]**cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs** -

Manual space calculation is prone to error

Description

Both `UserPosition` and `PredictionPool` manually calculate the required account space via the `calculate_space` function. Manual calculations increase the risk of future mismatches if fields change (e.g., new fields are added or different types are introduced).

Recommendation

Consider implementing Anchor's [`init\_space! macro`](#).

Status

Acknowledged

[Q-03]

`cfmm-prediction-market/programs/cfmm-prediction-market/tests/test_calculate_cfmm_sell_tokens.rs` - Test suite uses a custom approximation function instead of the actual on-chain logic

Description

The test suite defines its own helper function, `calculate_cfmm_sell_tokens`, to estimate sell payouts. This implementation is different from the actual logic used in `cfmm-prediction-market/programs/cfmm-prediction-market/src/math.rs:142`. For example, it utilizes a custom secant method solver and simplifications that are not guaranteed to match production behavior.

This is problematic because tests may pass with this approximation but fail in production, giving a false sense of correctness. Additionally, if protocol math changes (e.g., bug fix, refactor, fee handling), tests will not automatically detect regressions since they are not using the actual logic.

Recommendation

Consider importing and using the actual CFMM math functions directly in tests.

Status

Resolved

[Q-04]

cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs - Use constants instead of raw byte string seeds

Description

Throughout the program, PDA seeds are defined using inline byte strings instead of constants, i.e., `b"operator"`, `b"prediction-pool"`, and `b"user-position"`. This increases the difficulty of updating or refactoring PDA derivations due to the risk of typos.

Recommendation

Define constants for all seeds (e.g. `pub const OPERATOR_SEED: &[u8] = b"operator"`) and apply them across all the instructions.

Status

Resolved

[Q-05]

[cfmm-prediction-market/programs/cfmm-prediction-market/src/lib.rs#set_pool_status](#) - Use enum instead of raw u8 for pool status

Description

The pool's `status` field is currently stored as a raw u8 and updated without range checks via `prediction_pool.status = status`.

While this function is restricted to the operator, using raw integers instead of a strongly typed enum increases the risk of mistakes (e.g., accidentally setting the status to invalid values, such as 99).

Recommendation

Replace the raw u8 with an enum (e.g., `enum PoolStatus { Active, PreVerify, Resolved }`).

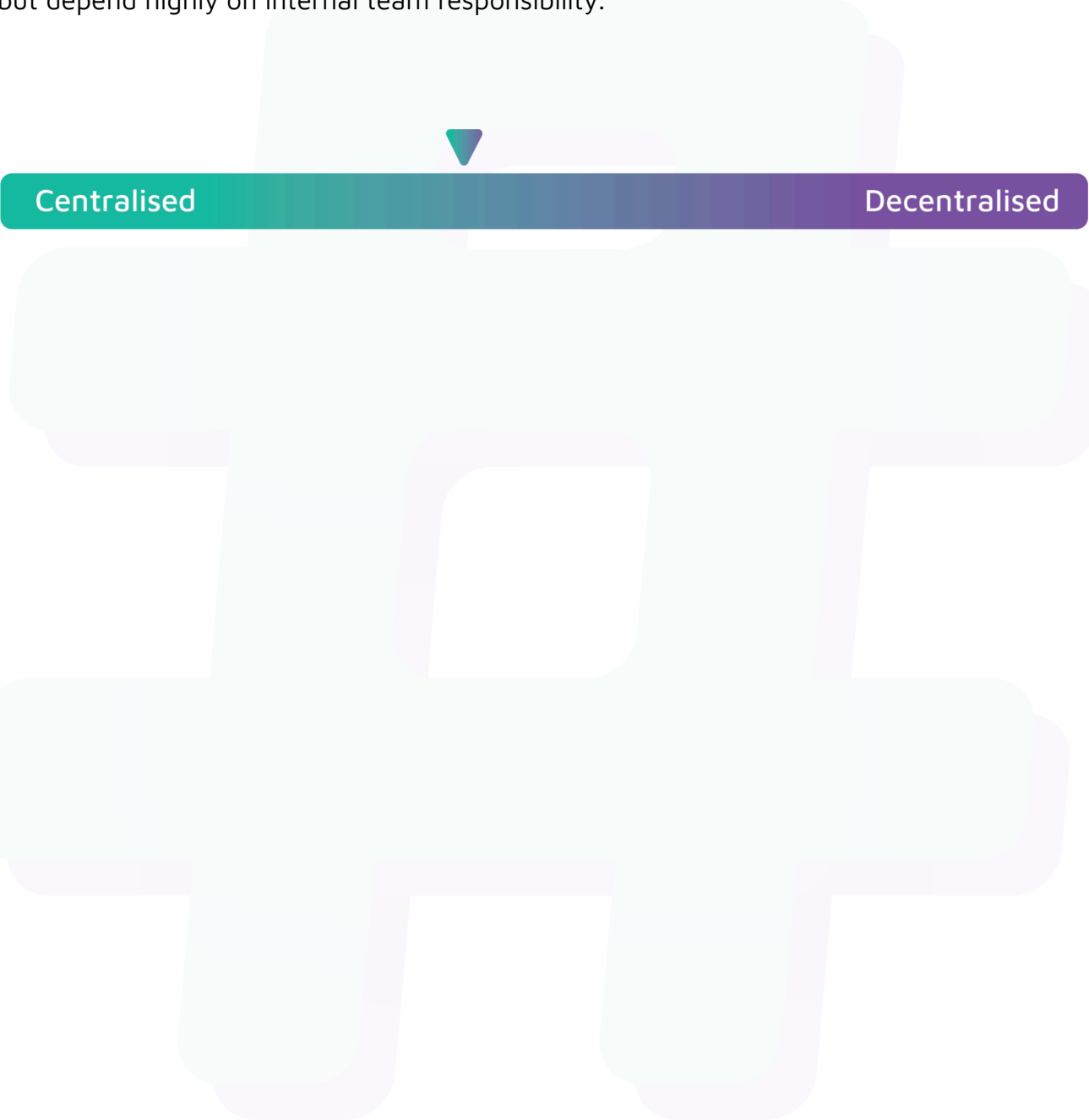
Status

Resolved

Centralisation

The ForeGate project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the ForeGate project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.